

Duplicate Receipt Detection

Design specification — portable to any expense/reimbursement system with file uploads and a reviewer role

Problem

An employee submits an expense with an attached receipt. Nothing in a typical expense system stops them from later submitting the same receipt again — by mistake, or to claim reimbursement twice. Reviewers have no way to know a receipt has already been used unless they happen to remember it.

Approach — two-tier detection

Run a fast, exact check first. Only fall through to a slower, fuzzier check if the fast one finds nothing. This keeps the common case (an identical file re-uploaded) cheap, and only pays the cost of the fuzzy check when it's actually needed.

Tier 1 — Exact hash (SHA-256)	Tier 2 — Perceptual hash (aHash)
Fingerprints the file's raw bytes.	Fingerprints what the image looks like.
Catches: the exact same file uploaded twice.	Catches: the same physical receipt photographed twice, re-compressed, or forwarded through WhatsApp.
Misses: any re-encode, crop, or re-photograph, even of the same receipt.	Slower, and only run when Tier 1 finds no match.

Flow

- 1

Upload
The submitter attaches a receipt file to their expense, exactly as they always do — no visible change to this step.
- 2

Compute exact hash
As soon as the file is available server-side, compute its SHA-256 hash from the raw file bytes.
- 3

Check for an exact match
Query the submitter's own past receipts for the same hash. Scope the comparison to that one submitter only — not the whole organization — to keep it fast. A match here is high-confidence: stop and flag.
- 4

No exact match? Compute a perceptual hash
Only if Tier 1 found nothing: resize the image to a small fixed grid, convert to greyscale, and derive an average-hash bit string from it.

5

Check for a near match

Compare the new perceptual hash against the submitter's past receipts' perceptual hashes using Hamming distance (the count of differing bits). A distance under a small threshold means the images are visually near-identical — flag as a probable duplicate.

6

Store the result silently

If either tier matched, set a duplicate flag (and which prior expense it matches) on the new expense record. Nothing is shown to the submitter — the expense saves normally.

7

Surface it to reviewers only

Wherever a reviewer (manager, HR, audit — whichever roles review expenses in this system) views the expense, show a flag such as “Possible duplicate — matches receipt from [date]” so they can look closer before approving.

Note: Comparison scope is deliberately self-only (each submitter checked only against their own history) rather than organization-wide, to keep both hash lookups fast as the receipt volume grows. Widen this only if the product need genuinely requires catching duplicates across different people.

Tech stack

Framed generically — swap in whatever your stack's equivalents are. No component here is unusual or requires a paid service.

Need	What it does	Typical choice
Exact hashing	SHA-256 over the raw file bytes	Any standard-library crypto/hash module (e.g. Node's built-in crypto, Python's hashlib) — no extra dependency needed
Image decode + resize	Needed to build the perceptual hash: shrink the image to a small grid (e.g. 8×8 or 16×16) and convert to greyscale	An image-processing library already common in most backends (e.g. sharp for Node, Pillow for Python)
Perceptual hash + comparison	Turn the resized/greyscale pixels into a bit string; compare two hashes by counting differing bits (Hamming distance)	Plain application code — simple bitwise logic, no library required
Storage	Two new fields on the expense record: an exact-hash column and a perceptual-hash column, both indexed per-submitter for fast lookup	Whatever the existing expense table already lives in (SQL or NoSQL both work fine)
Where the logic runs	Wherever receipt uploads are already handled server-side	Reuse the existing upload/receipt endpoint rather than standing up a new service

Data model additions

Two new fields on the existing expense/receipt record — no new table required:

```
receipt_sha256 text    - exact hash, hex-encoded
receipt_phash text     - perceptual hash, as a bit string or hex
duplicate_of_id uuid/id - nullable; points at the prior expense this one matches
```

Index both hash columns together with the submitter/user id, since every lookup is scoped to “this submitter's past receipts.”

UX rules

- The submitter sees nothing different — no warning, no blocked upload, no delay they'd notice. The check runs after the file is already accepted.
- Only reviewer-facing views (manager/approver, HR, audit — whatever roles review expenses) show the flag.
- The flag should link to (or at least name) the specific prior expense it matches, so a reviewer can compare the two directly rather than just being told “something's off.”
- An exact-hash match and a perceptual-hash match should probably read differently in the UI (e.g. “Identical file” vs. “Looks similar to”) since their confidence levels differ.